SPI Device
Nathaniel Ross

Task A: SPI Init

Do: Create C and Header files to initialize the SPI protocol.

Code:

spi_init.c:

```
void spi1_init(void)
{
	// configure clocks
	RCC->AHB1ENR |= (1 << 1);		// GPIOB clock enable
	RCC->AHB1ENR |= (1 << 2);		// GPIOC clock enable
	RCC->APB2ENR |= (1 << 12);		// SPI1 clock enable
	// configure PB3, PB4, PB5
	GPIOB->MODER &= ~(0b111111 << 6);	// clear PB3,PB4,PB5
	GPIOB->MODER |= (0b101010 << 6);	// AF mode for PB3,PB4,PB5
	GPIOB->AFR[0] &= ~(0xFFF << 12);	// clear AF selections
	GPIOB->AFR[0] |= (0x555 << 12);	// set all AF05
	GPIOB->OTYPER &= ~(0b111 << 3);	// push-pull output type
	GPIOB->PUPDR &= ~(0b111111 << 6);	// no pull-up, pull-down
	GPIOB->OSPEEDR |= (0b111111 << 6);	// high speed
	// configure PB6, PC7
	GPIOB->MODER &= ~(0b11 << 12);	// clear PB6
	GPIOC->MODER &= ~(0b11 << 14);	// clear PC7
	GPIOB->MODER |= (0b01 << 12);	// output mode for PB6
	GPIOC->MODER |= (0b01 << 14);	// output mode for PC7
	GPIOB->OTYPER &= ~(1 << 6);		// push-pull output type
	GPIOC->OTYPER &= ~(1 << 7);		// push-pull output type
	GPIOB->PUPDR &= ~(0b11 << 12);	// no pull-up, pull-down
	GPIOC->PUPDR &= ~(0b11 << 14);	// no pull-up, pull-down
	GPIOB->BSRR = (1 << 6);		// idle CS high
	GPIOC->BSRR = (1 << 7);		// idle RST high
	// configure master mode
	SPI1->CR1 = 0;			// clear configuration
	SPI1->CR1 |= (1 << 2);		// set master configuration
	SPI1->CR1 &= ~(0b111 << 3);
	SPI1->CR1 |= (0b011 << 3);		// baud rate = 84MHz/16 = 5.25 MHz
	SPI1->CR1 &= ~(1 << 1);		// CPOL = 0, clock idle low
	SPI1->CR1 &= ~(1 << 0);		// CPHA = 0, sample on first edge
	SPI1->CR1 &= ~(1 << 7);		// MSB transmitted first
	SPI1->CR1 &= ~(1 << 11);		// 8-bit data frame format
	SPI1->CR1 |= (1 << 9);		// software slave management
	SPI1->CR1 |= (1 << 8);		// internal NSS high (SSI)
	SPI1->CR1 |= (1 << 6);		// enable SPI
}
```

```
uint8_t spi1_transfer(uint8_t tx_data, uint8_t *rx_data)
{
	uint32_t timeout = 10000;
	while(!(SPI1->SR & (1 << 1)))		// wait for TXE to be set
	{
		if(timeout-- == 0)
			return SPI_ERROR;
	}
	*((volatile uint8_t*)&SPI1->DR) = tx_data;// write byte to data register
	timeout = 10000;
	while(!(SPI1->SR & (1 << 0)))		// wait for RXNE to be set
	{
		if(timeout-- == 0)
			return SPI_ERROR;
	}
	*rx_data = *((volatile uint8_t*)&SPI1->DR); // read received byte
	return SPI_OK;
}
uint8_t spi1_receive(uint8_t *rx_data)
{
	return spi1_transfer(0xFF, rx_data); // dummy byte to receive real byte
}
```

Results:

```
20:21:39 Build Finished. 0 errors, 0 warnings. (took 1s.7ms)
```

None so far, compiles correctly

Learned:

- How to initialize the SPI protocol using bare-metal programming in Embedded C
- Configuring the GPIO pins was very similar to configuring the I2C protocol: Enable the GPIOB/C clock, enable SPI1 clock, pins for alternate function mode, set the alternate functions based on the STM32F411xE datasheet, set push-pull
- The SPI protocol is different from I2C because it is **full-duplex** (master and slave send data to each other simultaneously) while I2C is **half-duplex** (master and slave take turns sending data).
- SPI pins use **push-pull** instead of **open-drain**. Push-pull is much faster than open-drain because it uses two transistors to drive HIGH and LOW, while open-drain uses one transistor to drive LOW and requires pull-up resistors to drive HIGH. SPI uses push-pull because it is used for high-speed data transfer (in our case, RF signals) and only connects to one device. To contrast, I2C uses open-drain because it can connect to many devices on the same line.
- The CS and RST pins for the SPI are regular GPIO pins, however they are updated using the BSRR (bit set reset register) because BSRR is **atomic** (basically instant). This is crucial in SPI communication, because data is transferring very fast, and if the chip select is not updated in time, a data transaction can be completely lost.
- The baud rate was set to 5.25 MHz (84Mhz/16) because the MFRC522 datasheet says its maximum communication rate with SPI is 10MHz
- CPOL (clock polarity) and CPHA (clock phase) were both set to zero. This means that the clock is LOW when idle, and the first data sample is on the first clock edge
- When transferring and receiving data, we have to wait for the transmit buffer to be empty and the receive buffer to be not empty. So even if there is no data to be sent, in order to receive data there has to be dummy values sent. This is part of the two-way communication between the Master and Slave.

Task B: Verify MFRC522 Device

Task: Initialize the MFRC522 device and read its Version Register to ensure the device is working.

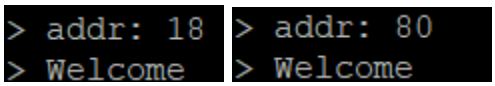
Code:

mfrc522.c:

```
#include "mfrc522.h"
#include <stdio.h>
uint8_t mfrc522_init(void)
{
	GPIOC->BSRR = (1 << 23);	// hold RST LOW
	HAL_Delay(1);
	GPIOC->BSRR = (1 << 7);		// set RST high
	HAL_Delay(50);			// allow oscillator to stabilize
	uint8_t result;
	uint8_t status = mfrc522_read_reg(VERSION_REG, &result);
	if(status == SPI_ERROR)
		return SPI_ERROR;
	// make sure chip is working
	if(result != 0x00 && result != 0xFF)
		return SPI_OK;

	return SPI_ERROR;
}
uint8_t mfrc522_read_reg(uint8_t reg, uint8_t *result)
{
	cs_low();				// pull CS low
	uint8_t rx_dummy;
	// transfer address byte
	// read: shift left, clear bit 0, set bit 7
	uint8_t status = spi1_transfer( ((reg << 1) & 0x7E) | 0x80, &rx_dummy);
	if(status == SPI_ERROR)
	{
		cs_high();			// pull CS high
		return SPI_ERROR;
	}
	// receive data byte
	status = spi1_receive(result);
	if(status == SPI_ERROR)
	{
		cs_high();			// pull CS high
		return SPI_ERROR;
	}
	cs_high();				// pull CS high
	return SPI_OK;
}
uint8_t mfrc522_write_reg(uint8_t reg, uint8_t value)
{
```

```
	cs_low();                               // pull CS low
	uint8_t rx_dummy;
	// transfer address byte
	// write: shift left, clear bits 0 and 7
	uint8_t status = spi1_transfer((reg << 1) & 0x7E, &rx_dummy);
	if(status == SPI_ERROR)
	{
		cs_high();
		return SPI_ERROR;
	}

	// send data byte
	status = spi1_transfer(value, &rx_dummy);
	if(status == SPI_ERROR)
	{
		cs_high();
		return SPI_ERROR;
	}
	cs_high();
	return SPI_OK;
}
```

Results:



Printing the VersionReg gave 0x18. This is different from the expected 0x91 or 0x92 because the version I have is likely a fake (I got it from an ELEGOO Arduino Kit). Importantly, it doesn't show 0x00/0xFF, and other key registers (ex: TxControlReg) return correct reset values (0x80).

Learned:

- How to use the SPI transfer and receive functions I made previously to write and read to the MFRC522
- When transferring the address byte, reading and writing have different protocols according to the MFRC522's datasheet: To read, shift the address left 1 bit, clear bit 0, and set bit 7. To write, shift the address left 1 bit and clear bits 0 and 7.
- CS has to be pulled low at the start of each read/write, and pulled high at the end of each read/write
- When initializing the MFRC522, RST has to be held LOW for at least 100ns to complete reset, then RST has to be set high with about a 40ms period to allow the internal oscillator to stabilize and the device to be configured. This is reflected in the mfrc522_init() function with a HAL_Delay(1) after RST LOW and a HAL_Delay(50) after RST HIGH.
- The MFRC522 I am using is likely a fake/copycat because its VersionReg (which is supposed to have reset value of 0x91/0x92) has a reset value of 0x18

Task C: Card UID Print

Do: Add a function to the main loop that prints the UID of a card when it is scanned by the MFRC522.

Code:

mfrc522.c:

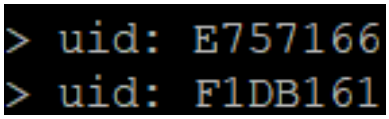
```
uint8_t mfrc522_read_uid(uint8_t *uid)
{
	uint8_t atqa;

	if(mfrc522_detect_card(&atqa) == SPI_ERROR)// send REQA byte
		return SPI_ERROR;

	mfrc522_write_reg(COMMAND_REG, 0x00);			// idle
	mfrc522_write_reg(COM_IRQ_REG, 0x7F);			// clear IRQ flags
	mfrc522_write_reg(FIFO_LEVEL_REG, 0x80);		// flush FIFO
	mfrc522_write_reg(FIFO_DATA_REG, 0x93);			// anticollision command
	mfrc522_write_reg(FIFO_DATA_REG, 0x20);			// anticollision command
	mfrc522_write_reg(COMMAND_REG, 0x0C);			// transceive
	mfrc522_write_reg(BITFRAME_REG, 0x00);			// full 8-bit frame
	mfrc522_write_reg(BITFRAME_REG, 0x80);			// start transmission

	uint32_t timeout = 10000;
	uint8_t val = 0;
	while(!(val & (1 << 5)))				// wait for RxIRq set
	{
		if(timeout-- == 0)
			return SPI_ERROR;
		if(mfrc522_read_reg(COM_IRQ_REG, &val) == SPI_ERROR)
			return SPI_ERROR;
	}

	uint8_t bcc;						// FIFO read
	for(int i = 0; i < 5; i++)
	{
		if(i < 4)
			mfrc522_read_reg(FIFO_DATA_REG, &uid[i]);
		else
			mfrc522_read_reg(FIFO_DATA_REG, &bcc);
	}

	if((uid[0] ^ uid[1] ^ uid[2] ^ uid[3]) != bcc) // bit check
		return SPI_ERROR;
	return SPI_OK;
}
```

```
uint8_t mfrc522_detect_card(uint8_t *atqa)
{
	mfrc522_write_reg(COMMAND_REG, 0x00);			// idle
	mfrc522_write_reg(COM_IRQ_REG, 0x7F);			// clear IRQ flags
	mfrc522_write_reg(FIFO_LEVEL_REG, 0x80);		// flush FIFO
	mfrc522_write_reg(FIFO_DATA_REG, 0x26);			// load REQA byte
	mfrc522_write_reg(COMMAND_REG, 0x0C);			// transceive
	mfrc522_write_reg(BITFRAME_REG, 0x87);			// start transmission

	uint32_t timeout = 10000;
	uint8_t val = 0;
	while(!(val & (1 << 5)))				// wait for RxIRq set
	{
		if(timeout-- == 0)
			return SPI_ERROR;
		if(mfrc522_read_reg(COM_IRQ_REG, &val) == SPI_ERROR)
			return SPI_ERROR;
	}

	if(mfrc522_read_reg(FIFO_DATA_REG, atqa) == SPI_ERROR)
		return SPI_ERROR;


	return SPI_OK;
}
```

main.c:

```
void handle_mfrc522(void)
{
	uint8_t uid[4];
	char msg[16];
	if(mfrc_state == 0 && mfrc522_read_uid(uid) == SPI_OK)
	{
		snprintf(msg, sizeof(msg), "> uid: ");
		HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
		for(int i = 0; i < 4; i++)
		{
			snprintf(msg, sizeof(msg), "%X", uid[i]);
			HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
		}
		snprintf(msg, sizeof(msg), "\r\n");
		HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
		mfrc_state = 1;
		last_detected_tick = HAL_GetTick();
	}
	else if(mfrc_state == 1 && HAL_GetTick() - last_detected_tick > 2000)
		mfrc_state = 0;
}
```

Results:



Blue card first, white card second

Learned:

- How to detect a card scan and read the UID of a card for the MFRC522. To do this, I had to read through the documentation of the MFRC522 and determine which device registers needed to be altered. The process of doing so was specific to the MFRC, and it involved clearing the command register, clearing interrupt requests, flushing the device's FIFO queue, loading the REQA (request guard) byte, starting the transmission, and then receiving data from the device.
- The process of transmitting and receiving from the MFRC522 followed the ISO 14443 / RFID protocol, with the use of REQA, ATQA, anticollision, and UID. In the "detect_card" function, the ATQA was found to verify a reading.
- In all, learning this protocol involved reading the datasheet, understanding registers, building a driver for this device, and verifying each step as I went
- Handling the card scanning in the main loop required **debouncing**. To explain, when a card would be tapped to the sensor, it would initially register dozens of scans per second. This is not the intended behavior, so debouncing was implemented in software. This debouncing followed a state machine of the card's state, with state 0 being a scan and 1 being no scan. A counter was implemented that, if there are at least 2000 clock ticks between scans, the scan is valid. This removed the console being flooded with scans and registers one scan every 2 seconds.

Task D: Card Authentication

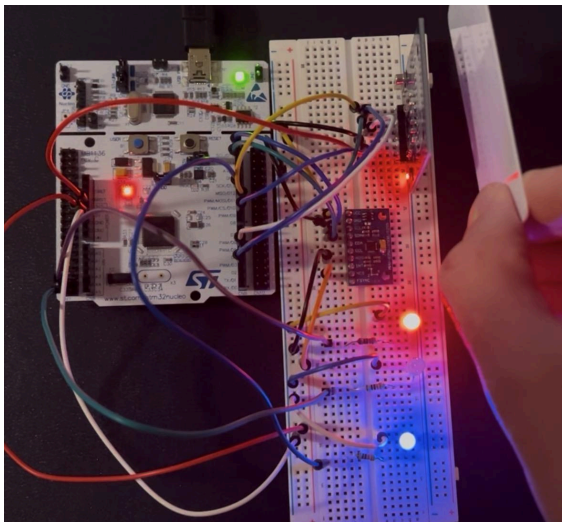
Do: Add functionality that turns on a green LED when the correct card is scanned and turns on a red LED when the incorrect card is scanned.

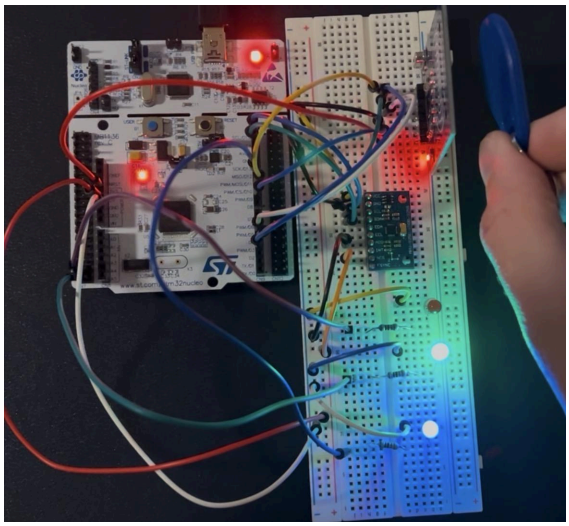
Code:

main.c:

```
void check_uid(uint8_t *uid)
{
	char msg[64];
	if(memcmp(uid, correct_uid, 4) == 0)
	{
		GPIOC->ODR &= ~(1 << 2);		// turn red led off
		GPIOC->ODR |= (1 << 3);		// turn green led on
		snprintf(msg, sizeof(msg), "> Authentication Successful\r\n");
		HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

	}
	else
	{
		GPIOC->ODR &= ~(1 << 3);		// turn green led off
		GPIOC->ODR |= (1 << 2);		// turn red led on
		snprintf(msg, sizeof(msg), "> Authentication Denied\r\n");
		HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
	}
}
```

Results:





```
> Authentication Denied
> Authentication Successful
auth_leds_off
> Authorization LEDs off
```

I also added a command to turn off the LEDs because, once a card is scanned, at least one is on at all times.

Learned:

- This part of the project reinforced basic embedded systems concepts. This means configuring the GPIO pins for the red and green LEDs, wiring them correctly with current-limiting resistors, and using pointers correctly for passing variables.
- A small bug was fixed: when the user typed the backspace in the UART Shell, the backspace character would become part of the command, resulting in an “unknown command” error. This is because the software checked for “\b”, while the terminal sent “0x7F” (delete) instead. Now both cases are accounted for.

What was learned overall:

- Basics of the SPI communication was learned and implemented in this project. SPI is a **full-duplex** protocol (communicating both ways), different from I2C which is half-duplex. SPI pins use **push-pull** instead of open drain because push-pull is much faster. This is good for high-speed data transfer, such as the RF device used in this project.
- Basic embedded system concepts were reinforced, such as configuring clocks, configuring GPIO pins for intended behavior, reading datasheets to learn intended behavior, and setting or clearing bits in registers.
- For SPI communication, because it is both ways (master to slave and slave to master), there sometimes has to be dummy information sent one way in order to receive information from the other
- Setting up the MFRC522 was unique to the information found in its datasheet, however it did follow the ISO 14443 / RFID protocol with the use of REQA, ATQA, anticollision, and UID.
- **Debouncing** was used for the card scanning feature. Instead of a card being read dozens of times per scan, a counter and state machine was implemented to have only one read per card scan.
- Overall, this project reinforced embedded systems topics covered across all four projects so far. This includes configuring GPIO pins, bit shifting to configure registers, using pointers, and following the STM32 F411RE datasheet and datasheets of devices added to this project. Further improvements of this project could be better pin and breadboard management. Currently, the wiring is fairly jumbled and the breadboard is slightly disorganized. In the future, these pins and layout could be thought out before implementation began. There is still potential to add more devices or protocols to this project, however further work will focus on using FreeRTOS with the current implementation.